AN EBPF-BASED BEHAVIORAL DETECTION AND MULTI-AGENT CONTAINMENT ARCHITECTURE FOR LINUX SERVER WORKLOADS

A PREPRINT

Pardhu Sri Rushi Varma Konduru
Dept. of Cybersecurity, ECE & IoT
Malla Reddy University, Hyderabad
2311CS040089@mallareddyuniversity.ac.in

Tejaswini Kandukoori
Dept. of Cybersecurity, ECE & IoT
Malla Reddy University, Hyderabad
2311CS040077@mallareddyuniversity.ac.in

Karthik Kanne
Dept. of Cybersecurity, ECE & IoT
Malla Reddy University, Hyderabad
2311CS040080@mallareddyuniversity.ac.in

Rupa Yeshvitha Karedla
Dept. of Cybersecurity, ECE & IoT
Malla Reddy University, Hyderabad
2311CS040082@mallareddyuniversity.ac.in

August 3, 2026

ABSTRACT

Kernel Borderlands (KB) is a kernel-level runtime defense framework for Linux composed of five subsystems: an eBPF-based Ring-0 sensor (`kb-core`), a Go control plane for scoring, storage, and enforcement (`kb-control-plane`), a Python multi-agent swarm for adaptive response (`kb-aads`), a stateless Rust integrity watchdog (`kb-checker`), and a set of operator interfaces (`kb-op`). This paper documents the design, implementation history, and current status of each subsystem in turn — including defects found and fixed along the way, and defects still open — rather than concentrating on any single component. Substantial, independently verifiable engineering effort went into each subsystem: kernel-verified enforcement primitives and authorization gates in `kb-core`; a two-tier storage engine and a hash-chained, verifiably tamper-evident audit log in `kb-control-plane`; a Ray-based actor swarm in `kb-aads`, of which one role — Containment — now carries a trained and live-verified reinforcement-learning policy; a stateless boot-gating watchdog in `kb-checker`; and four operator-facing interfaces in `kb-op`. We additionally report a source-level security audit conducted across all four non-RL subsystems, surfacing fifteen open defects, three of them critical, alongside three previously undocumented fixes. Where a subsystem or feature is incomplete or stubbed at the time of writing, we say so explicitly rather than omitting it.

Keywords eBPF · Linux security · runtime containment · multi-agent systems · reinforcement learning · intrusion detection · distributed systems

Code availability. The full source referenced throughout this paper is available at `https://github.com/pardhuvarmax/kernel-borderlands` [Kernel Borderlands Team, 2026].

1 Introduction

Runtime defense for Linux hosts increasingly relies on kernel-level observability: extended Berkeley Packet Filter (eBPF) programs attached to LSM hooks, tracepoints, and kprobes observe process behavior at Ring 0 with low overhead [Gregg, 2019, Vieira et al., 2020], building on packet-filtering foundations dating back to the original BSD Packet Filter [McCanne and Jacobson, 1993]. Observability alone is not response — a runtime defense system must also decide, act, and be able to prove that its own decision trail has not been tampered with after the fact. Kernel Borderlands (KB) [Kernel Borderlands Team, 2026] is organized around this observe-score-decide-enforce-audit loop, implemented

across five cooperating subsystems, each of which represents an independent, substantial body of engineering work: `kb-core` (Section 2), `kb-control-plane` (Section 3), `kb-aads` (Section 4), `kb-checker` (Section 5), and `kb-op` (Section 6).

This paper's aim is to document KB as it currently stands, subsystem by subsystem, including what each subsystem does, notable defects discovered and corrected during its development, and — honestly — what remains incomplete. Section 7 reports a source-level security audit conducted across four of the five subsystems. Section 8 summarizes per-subsystem status in one table. We do not attempt to make any single subsystem the centerpiece of this report; each received comparable engineering effort and is treated accordingly.

2 kb-core: eBPF Sensor and Enforcement Primitives

`kb-core` is a C/eBPF subsystem, built with CO-RE (Compile Once, Run Everywhere) via BTF for cross-kernel portability without per-kernel-version recompilation. It is the only subsystem with a presence in Ring 0.

2.1 Hook Points and the Containment Model

`kb-core` attaches LSM hooks and tracepoints covering process execution (`bprm_check_security`), file access (`file_open`), network connect/bind, memory-protection changes (`file_mprotect`), and credential changes (`commit_creds`), and maintains a five-level containment state per monitored process, uniform across all hooks, escalating through the standard Linux primitives of cgroups, seccomp, and namespaces [Kerrisk, 2010]: `NONE` (no entry; every hook passes through), `CGROUP` (resource throttling only, no LSM blocking), `SECCOMP` (blocks new network connections/binds and process exec), `NAMESPACE` (adds RWX/exec memory-protection blocking on top of `SECCOMP`), and `TERMINATE` (blocks everything interceptable, combined with a userspace `SIGKILL` as a belt-and-suspenders measure). Every hook returns a uniform `-EPERM` rather than hook-specific error codes, so the kernel presents a consistent "Operation not permitted" regardless of which containment layer intervened.

2.2 CPM and CWP: Authorization Gates on Containment Writes

Two authorization layers sit in front of containment map writes, both implemented in this project's current development cycle. **CPM (Critical Process Module)** refuses to contain PID 1, kernel threads, self-registered sensor components, or a protected-executable registry. The classifier (`cpm_classify()`) is deliberately implemented in userspace C rather than in-kernel eBPF, since that is where the actual containment-map write occurs in this codebase — a documented deviation from an earlier design sketch that placed the gate purely in-kernel. Two eBPF maps (`protected_pids_map`, `protected_exec_paths_map`) back it, populated at exec time and cleared at exit time so that a reused PID cannot inherit a predecessor's protection. **CWP (Critical Workload Protection)** extends the same idea to a broader class of protected workloads, identified by either path or SHA-256 content hash (`protected_workloads_map`, sized to 8192 entries to comfortably exceed realistic protected-workload counts in microservice-scale deployments), evaluated strictly after CPM and never merged with it. Both gates were verified against the live running sensor, not merely built: a live test confirmed that a containment command for a protected PID was rejected correctly while the connection was simultaneously saturated with unrelated telemetry.

2.3 The kbd.sock / kbct.sock Split

Telemetry and containment commands were originally multiplexed on a single, hand-rolled socket connection between the sensor and the control plane. This proved to be a latent availability bug: a telemetry-volume burst could fill the connection's send buffer, `write()` would return `EAGAIN`, and the sensor's error handling misread this as a dead connection and closed it — taking containment delivery down as collateral damage of an unrelated telemetry spike. The fix splits control traffic (containment commands, sensitive-path and rules pushes) onto a dedicated `kbct.sock`, leaving `kbd.sock` telemetry-only, and additionally ignores `SIGPIPE` process-wide so a closed peer cannot kill the sensor via signal regardless of the split. A secondary, previously-latent bug was found while wiring this up: the containment-command read path's error handler unconditionally reset a global file descriptor rather than the one actually passed in.

2.4 Rate Limiting

Telemetry rate limiting uses per-PID/per-resource hybrid token buckets implemented with BPF task-local storage and LRU hash maps, tracking limits by `PID + Resource ID` (e.g., destination IP, syscall ID) specifically to prevent an evasion technique in which an attacker distributes activity across many distinct resources to stay under a single flat

per-PID limit. Tier-1 events — privilege escalation, LSM denials, TLS-plaintext writes — are permanently exempted from throttling by design: the busier or more heavily attacked a system is, the more important it is that these specific events are never silently dropped for volume reasons.

2.5 Known Defects

A historical, critical defect is documented here for completeness: an early version of the sensitive-path LSM block was unconditional (applied to every process, not only contained ones), and the first time it fired correctly in the project's history it blocked `/etc/sudoers`/`/etc/shadow` reads system-wide, breaking `sudo`, `su`, `pkexec`, and login authentication simultaneously on the test host, recoverable only via a full external VM restart. This was fixed by making the block containment-gated, consistent with every other hook. A related bug — the block technically never enforced anything even when intended to, because `bpf_d_path()` does not zero the tail of its output buffer past the NUL terminator, causing exact-match lookups against a fixed-size key to silently fail — was found and fixed by explicitly zero-filling the buffer. Two defects remain open at the time of writing, found during the audit in Section 7: the level-3+ "block all file opens" guarantee fails open if `bpf_d_path()` itself fails before the containment-level check runs, and three PID-keyed telemetry maps (syscall totals/counts, credential snapshots) are never cleaned up on process exit, unlike their CPM/CWP counterparts, risking silent map exhaustion under long-running PID churn.

3 kb-control-plane: Scoring, Storage, and Enforcement

`kb-control-plane` is a Go daemon (`kbd`) responsible for ingesting sensor telemetry, maintaining process state, exposing a gRPC and HTTP API, and issuing containment commands back to the sensor.

3.1 Storage: L1/L2 Hybrid

State is kept in a two-layer store, per this project's ADR-1: an in-memory `sync.Map` (L1), authoritative for all reads, and an asynchronously-written SQLite database in WAL mode (L2), durable and single-writer (`SetMaxOpenConns(1)`) via a buffered channel, so that L1 writers never block on disk I/O even under load. On startup, L1 is restored from L2's durable copy. PID-reuse safety is handled explicitly: a process-exit wire message flushes both the stale L1 entry and the L2 row immediately, so a later, unrelated process reusing the same PID cannot inherit a predecessor's cached state.

3.2 The Audit Log

Every enforcement action, policy reload, and agent decision is recorded in a SHA-256 hash-chained append-only log (`entry_hash = SHA256(fields || prev_hash)`), such that mutating or deleting any row breaks every hash computed after it. A `VerifyChain()` routine existed correctly for some time before being wired to anything callable outside its own test — meaning the security property existed in code but delivered no operational value until an HTTP route (`GET /api/audit/verify`) and a gRPC `VerifyAuditChain` method were added to actually invoke it, alongside a `kbctl audit verify` command (Section 6). This audit's own review of `VerifyChain()` found that it currently discards the error returned by its SQL row-scan step, which could produce a false "chain broken" verdict on an ordinary I/O or schema error rather than a real tamper event — reported as an open defect in Section 7.

3.3 Policy Engine and Hot-Reload

`policy.yaml` configures per-process auto-terminate overrides and the sensitive-paths list consulted by `kb-core`. Changing this configuration originally required a full daemon restart, which also dropped every sensor connection and forced a cold-start L1 rebuild. This was corrected by adding a `ReloadPolicy` gRPC method that re-reads `policy.yaml` and atomically swaps a freshly built `policy.Engine` into place under a `sync.RWMutex` — a full lock on the (rare) write path, a read lock on the (frequent, per-zone-transition) read path — verified during this audit to be genuinely race-free, not merely apparently working.

3.4 Known Defects

A historical, critical defect: the wire-level constant for containment commands (`MsgTypeContainmentCmd`) was 3 on the Go side but the C sensor checked for 5, so every containment command issued via any operator interface was silently dropped, while the control plane's own audit log recorded a successful entry regardless (the sensor never NACKs) — completely masking the failure until a live test against the running sensor exposed it. Open defects found during this audit (Section 7) include an HTTP API bound to all network interfaces with no authentication (the most severe finding

of the audit), a telemetry-ingestion bug that silently resets an actively contained process's recorded state back to `NONE`, the audit-chain scan-error issue above, absent input validation/rate limiting on containment RPCs, and an SSH operator listener with no brute-force throttling and a dev-mode fallback that can silently weaken key material.

4 kb-aads: The Agent Defense Swarm

`kb-aads` is a Python subsystem built on Ray, organized as a set of role-specialized actors intended to jointly monitor, investigate, and respond to threats.

4.1 Actor Architecture

Roles are implemented as Ray actors subclassing a common `BaseAgent`: **Patroller** (baseline monitoring), **Hunter** (threat investigation), **Healer** (false-positive recovery), **Containment** (enforcement-level selection), and a **Judge/Jury/Executor (JJE)** consensus mechanism intended to gate high-stakes decisions behind a quorum vote before the **Executor** carries them out over gRPC to `kb-control-plane`. The actor scaffolding itself required a non-trivial fix early in its development: `BaseAgent` was originally decorated `@ray.remote` directly, which raises `ActorClassInheritanceException` the instant any concrete role subclass also applies `@ray.remote` — meaning no concrete role actor could actually be constructed under the original design. The fix removes the decorator from the base class and applies it individually per concrete subclass (with a `RemoteBaseAgent` convenience wrapper for roles with no dedicated subclass), after which the swarm could be verified to actually spawn, tick, and report status end to end.

4.2 JJE Consensus: Design Intent and Current Gaps

The JJE consensus design calls for Judge to open a round when a Patroller-raised alert crosses a threshold, a dynamically-spawned Jury pool to cast weighted votes, and Executor to carry out the resulting decision only after quorum. As implemented, **Jury**'s vote condition compares an alert's confidence value against a threshold of 75.0, while every confidence value elsewhere in the system — the control plane's own `SubmitAgentDecision` threshold, and the wire-level `confidence` field itself — is a 0.0–1.0 float. Under the scale used everywhere else, this condition cannot be true for any realistic payload, making Jury's containment vote unreachable in practice; `coordinate_consensus`'s pool size is additionally hardcoded rather than read from its own configuration file, and a crashed juror's exception is not caught, so one failing actor can abort an entire consensus round rather than degrading gracefully. These are reported as open defects in Section 7 rather than fixed in this cycle, since correcting Jury's decision logic is properly scoped together with giving it an actual trained policy, which has not yet been done (only Containment has, per Section 4.3).

4.3 Containment: A Trained Reinforcement-Learning Policy

Of `kb-aads`'s decision-making roles, Containment is the only one currently backed by a trained policy rather than a stub or an unreachable rule. Applying reinforcement learning to intrusion response specifically (as opposed to detection alone, the more heavily studied problem [Liao et al., 2013]) is an active area [Nguyen and Reddi, 2021]; we contribute a single, concrete, reproducible instance of it rather than a general treatment. We formulate its decision as a single-step reinforcement learning problem: a five-way discrete action (`ContainmentLevel`, matching Section 2's enumeration) over a 5-dimensional observation built from the same fields KB's control plane already carries on the wire for a process (`score`, `zone`, a binary `uid_is_root` flag, `score_delta`, and an `event_type` code), with reward

$$r(a,t) = 1 - 0.3\,|a-t| - 0.2\,|a-t| \cdot \mathbb{1}[a < t] \tag{1}$$

penalizing under-containment ($a < t$) more heavily than over-containment relative to the labeled target level t.

Data. No real eBPF-captured attack corpus existed for KB at the time of this work, so real public data matching KB's actual observation surface — Linux syscall sequences, not network flow, which an initially-tried NSL-KDD dataset was rejected for mismatching — was used instead. We mapped five of ADFA-LD's six attack categories [Creech and Hu, 2013, 2014] onto five of KB's own named attack scenarios (Table 1); KB's `memory_exploit` scenario has no ADFA-LD equivalent and is not covered. The benign class combines ADFA-LD's Normal traces with real `/proc`-derived telemetry sampled from a live development host.

Training and a mid-training defect. We trained a PPO policy [Schulman et al., 2017] via RLlib [Liang et al., 2018] on Ray [Moritz et al., 2018]. An initial run reached 97.8% overall test accuracy but only 60–73% on the

Table 1: ADFA-LD category to KB attack-scenario mapping.

ADFA-LD category	KB scenario
Adduser	privilege_escalation
Hydra_FTP	credential_access
Hydra_SSH	lateral_movement
Java_Meterpreter	process_injection
Meterpreter, Web_Shell	reverse_shell

`CGROUP/SECCOMP` classes (Table 2), root-caused to `credential_access` and `lateral_movement` sharing the same engineered `event_type` code and `zone` floor. Assigning each category a distinct code and retraining improved every attack category to 97–100% and reduced the under-containment rate to 0.0%, at the cost of `NONE` accuracy falling from 100% to 82.2% (Table 3) — a trade we consider correct for a containment system, since a false positive on benign traffic costs a review action while a false negative on a real threat costs a missed compromise.

Table 2: Per-level test accuracy before and after the event_type fix.

Level	Before fix	After fix
NONE	100.0%	82.2%
CGROUP	60.0%	100.0%
SECCOMP	70.4%	100.0%
NAMESPACE	100.0%	97.1%
TERMINATE	93.3%	100.0%

Table 3: Overall test-set results before and after the fix.

	Before fix	After fix
Overall accuracy	97.8%	84.4%
Under-containment rate	1.1%	**0.0%**

Live verification. We verified the trained checkpoint three ways: offline test-set evaluation (above); a direct unit-level check against a live `ContainmentAgent` Ray actor for one synthetic observation per attack scenario; and an end-to-end run streaming real telemetry from KB's actual eBPF sensor and control plane on a live host. The end-to-end check surfaced an integration defect distinct from the model: the control plane returns a zero-value default `ProcessState` (not an error) for an unscored PID, and a naive feature mapping misread the resulting default `uid=0` as "running as root," producing spurious high-severity decisions for untracked, usually short-lived processes — corrected by treating an empty `comm` field as the true not-found signal.

4.4 Status Note

Patroller, Hunter, and Healer remain non-functional stubs at the time of writing: Patroller's `tick()` sets a static status string, Hunter's investigation method contains only unimplemented TODOs, and Healer performs no recovery logic. No trained or rule-based decision currently flows through these roles into a real enforcement action.

5 kb-checker: Stateless Integrity Watchdog

`kb-checker` is a Rust subsystem deliberately kept stateless and network-free beyond its own local diagnostic socket, following a KISS design principle intended to minimize its own attack surface as the component responsible for detecting compromise of everything else.

5.1 Active Auditing

`kb-checker` performs three classes of active audit rather than passive log-watching. **Bytecode integrity**: it retrieves loaded BPF program metadata from the kernel, extracts the JIT-compiled instruction footprint via `bpf_obj_get_info_by_fd`, and computes a SHA-256 hash to verify it against an expected signature. **Map self-healing**: it traverses

the live kernel containment map via `libbpf-sys`, cross-references its keys against the control plane's database, and heals discrepancies with `bpf_map_update_elem`. **Liveness**: it connects to the control plane's telemetry stream, forks a disposable test process (`/bin/true`), and verifies that the corresponding `sched_process_exec` event is observed within a bounded (3-second) window, catching a fully wedged sensor that is still technically running but no longer actually emitting telemetry. Execution-latency audits additionally read JIT instruction run-count and cumulative run-time counters to flag hooks whose average execution time exceeds a 500ns threshold.

5.2 Boot-Sequence Gating

`kb-checker` gates the swarm's ability to start at all. Per the project's boot-sequence specification, systemd is configured such that if `kb-checker`'s audits fail, the Ray-based agent service (`kbd-agent.service`) is blocked from starting, effectively quarantining a host from executing swarm workloads until the watchdog is satisfied that the layers beneath it are intact.

5.3 Fallback Containment Hierarchy

For cases where the primary daemons cannot be cleanly unloaded via normal systemd control, `kb-checker` implements a three-layer hard fallback: force-killing the sensor's userspace process group (`SIGKILL`), detaching dangling BPF hooks directly via `bpftool` if the process-level kill is insufficient, and, as a last resort, an `iptables`-based network quarantine that drops all traffic except loopback, preserving local console access for an operator to intervene manually.

5.4 Status Note

`kb-checker` was not found to have any open defects within the time available for the audit in Section 7; this reflects the scope of time spent on it during this audit cycle, not a formal claim of exhaustive verification.

6 kb-op: Operator Interfaces

`kb-op` provides four operator-facing surfaces over KB's gRPC and HTTP APIs.

6.1 kbctl

`kbctl` is a Go CLI dialing `kbd`'s gRPC service directly over its Unix domain socket. It supports process isolation and restore (`process isolate`, with an explicit `CGROUP|SECCOMP|NAMESPACE|TERMINATE` level flag), zone classification overrides, audit-chain verification and JSON export (`audit verify`, `audit export`), policy hot-reload (`policy reload`, Section 3), and system statistics. An open defect found during this audit: `audit verify` calls `os.Exit(1)` directly on a detected chain break, bypassing the function's own deferred connection-cleanup calls — minor in practice for a short-lived CLI process, but notably sloppy in exactly the one command whose entire purpose is flagging tampering.

6.2 kb-tui

`kb-tui` is a Rust/ratatui terminal console, reachable over an SSH listener hosted by `kbd` itself (`internal/ssh/`), which spawns `kb-tui` as a subprocess connected to the SSH session's PTY after public-key authentication. The project's own architecture documentation records a considered, not-yet-implemented decision to replace this custom Go SSH server with a real OpenSSH instance plus a forced command, specifically because a hand-rolled SSH implementation carries a far thinner security track record than ~25 years of OpenSSH hardening for the one listener in the system that is network-exposed at all. Two related open defects were found during this audit: no rate limiting on failed authentication attempts, and a development-mode fallback path that can silently relocate host key material relative to the current working directory rather than a fixed location.

6.3 kb-dashboard

`kb-dashboard` is a TypeScript web frontend against `kbd`'s HTTP API. This audit found its isolate/restore controls to be the client side of the most severe defect surfaced in this work (Section 7): the HTTP endpoints they call have no authentication and are served with a wildcard CORS header, and the dashboard additionally hardcodes its API host to `localhost:8080`, which would make that exposure network-reachable rather than local-only the moment the hardcoded host is changed for a non-local deployment without the underlying API also being fixed.

6.4 kb-mcp

`kb-mcp` exposes KB's telemetry and query surface as a Model Context Protocol server, intended to let LLM-based tooling query process state and zone classifications through the same gRPC API `kbctl` and `kb-tui` use. This subsystem was not included in the source-level audit reported in Section 7 within the time available; we note this explicitly rather than imply coverage we did not perform.

7 Security Audit

Independent of any single subsystem's feature work, we conducted a source-level audit across `kb-core`, `kb-control-plane`, `kb-aads`, and `kb-op` (`kb-checker` was included in scope but no defects were found within the time available, and `kb-mcp` specifically was not covered), verified directly against current source rather than taken on prior documentation's word. The audit surfaced fifteen confirmed open defects (the five most severe listed in Table 4; the remainder are discussed alongside their respective subsystems above), three previously-undocumented resolved items, and reconfirmed three historical critical bugs already fixed earlier in the project's history and discussed in Sections 2–3. The single most severe finding is BUG-001: an HTTP endpoint bound to all network interfaces with no authentication and a wildcard CORS header, capable of terminating or restoring containment on an arbitrary process ID from any network-reachable origin.

Table 4: The five most severe open findings from the security audit.

ID	Severity	Summary
BUG-001	Critical	Unauthenticated, wildcard-CORS HTTP containment API (§3, §6)
BUG-002	Critical	Telemetry update silently resets active containment to NONE (§3)
BUG-003	High	Consensus vote threshold uses wrong confidence scale, dead code (§4)
BUG-004	High	LSM hook fails open on path-resolution failure (§2)
BUG-005	Medium-High	Audit-chain verifier discards its own scan error (§3)

8 Current Project Status

Table 5 summarizes what is complete, partial, or outstanding across all five subsystems at the time of writing.

Table 5: Per-subsystem implementation status.

Subsystem	Status
kb-core (§2)	Complete for the enforcement primitives documented here (containment levels, CPM/CWP gates, rate limiting, socket split), each independently live-verified. Two open defects found in this audit.
kb-control-plane (§3)	Complete for storage, scoring ingestion, gRPC/HTTP API, audit-chain logging, and policy hot-reload. Five open defects found in this audit, two critical.
kb-aads (§4)	Actor scaffolding functional; Containment role trained and live-verified. Patroller, Hunter, Healer, and Judge/Jury consensus remain non-functional stubs; Jury's vote condition is additionally confirmed unreachable under realistic input.
kb-checker (§5)	Complete for its stated scope (stateless integrity auditing, boot-sequence gating, fallback containment). No defects found in this audit within the time available.
kb-op (§6)	`kbctl`, `kb-tui`, and `kb-dashboard` functional against the current gRPC/HTTP surface, one defect found in each. `kb-mcp` functional per its own scope but not included in this audit's coverage.

9 Limitations and Future Work

Several gaps are explicitly acknowledged rather than papered over. KB's own attack-lab data pipeline remains unbuilt, so Containment's training data (Section 4.3) is a public-dataset stand-in with no coverage for the `memory_exploit`

scenario, and no mapping yet exists from `kb-core`'s real eBPF event-type vocabulary to the category codes that policy was trained on. Jury, Healer, and Hunter require both their underlying logic defects fixed (Section 4) and their own training or implementation work before they can contribute real decisions. `kb-mcp` was not covered by this audit cycle. Most importantly, the security findings in Section 7 — particularly the unauthenticated containment endpoint and the containment-state reset bug — should be treated as blocking issues for any deployment beyond a development host, independent of feature completeness elsewhere in the system.

10 Conclusion

We documented Kernel Borderlands subsystem by subsystem: kernel-verified enforcement primitives and authorization gates in `kb-core`; a two-tier storage engine, hash-chained audit log, and hot-reloadable policy engine in `kb-control-plane`; an actor-based swarm in `kb-aads` carrying one trained and live-verified reinforcement-learning decision role; a stateless boot-gating watchdog in `kb-checker`; and four operator interfaces in `kb-op`. Substantial, independently verifiable work underlies each of these, and each was subjected to the same source-level scrutiny in this paper's security audit, which surfaced a prioritized, concrete list of defects — most notably a critical unauthenticated containment endpoint — representing the necessary next work before any subsystem should be considered production-ready.

References

Kernel Borderlands Team. Kernel borderlands: Kernel-level runtime defense framework. `https://github.com/pardhuvarmax/kernel-borderlands`, 2026.

Brendan Gregg. BPF performance tools. In *Addison-Wesley Professional Computing Series*, 2019.

Marcos AM Vieira, Matheus S Castanho, Racyus DG Pacifico, Elerson RS Santos, Guilherme PR Junior, and Luiz FM Vieira. Fast packet processing with eBPF and XDP: Concepts, code, challenges, and applications. *ACM Computing Surveys (CSUR)*, 53(1):1–36, 2020.

Steven McCanne and Van Jacobson. The BSD packet filter: A new architecture for user-level packet capture. In *USENIX Winter 1993 Conference (USENIX Winter 1993)*, 1993.

Michael Kerrisk. *The Linux Programming Interface: A Linux and UNIX System Programming Handbook*. No Starch Press, 2010.

Hung-Jen Liao, Chun-Hung Richard Lin, Ying-Chih Lin, and Kuang-Yuan Tung. Intrusion detection system: A comprehensive review. *Journal of Network and Computer Applications*, 36(1):16–24, 2013.

Thanh Thi Nguyen and Vijay Janapa Reddi. Deep reinforcement learning for cyber security. *IEEE Transactions on Neural Networks and Learning Systems*, 2021.

Gideon Creech and Jiankun Hu. Generation of a new ids test dataset: Time to retire the kdd collection. *2013 IEEE Wireless Communications and Networking Conference (WCNC)*, pages 4487–4492, 2013.

Gideon Creech and Jiankun Hu. A semantic approach to host-based intrusion detection systems using contiguous and discontiguous system call patterns. *IEEE Transactions on Computers*, 63(4):807–819, 2014.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

Eric Liang, Richard Liaw, Robert Nishihara, Philipp Moritz, Roy Fox, Ken Goldberg, Joseph Gonzalez, Michael Jordan, and Ion Stoica. RLlib: Abstractions for distributed reinforcement learning. In *International Conference on Machine Learning*, pages 3053–3062. PMLR, 2018.

Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elibol, Zongheng Yang, William Paul, Michael I Jordan, and Ion Stoica. Ray: A distributed framework for emerging AI applications. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, pages 561–577, 2018.